AgriSwarm: Technical Report

AMD AI DevMaster Hackathon, Track 3 (Physical AI)

A drone swarm that takes an order in plain English. If it cannot see the plant it was asked to remove, it teaches itself to see it in about two minutes and shares what it learned with the other drones.

Built and measured on one **AMD Radeon PRO W7900** (48 GB) with ROCm 7.2, in three days by a team of three. Every number here came off that machine.

1. The problem

When a farmer sprays for weeds, the whole field gets sprayed. The machine cannot tell a weed from a crop, so it treats everything. The cost and the runoff scale with the ground you cover, not with the weeds actually out there.

AgriSwarm sprays weeds one at a time. You type an order. The drones split the field between themselves, fly a survey at 8 m, spot weeds with a downward camera, and drop to 1.2 m to spray each one.

The idea worth stealing: **a robot should be able to gain new vision while it is working.** Farm robots ship with a fixed list of things they recognise, decided months before anyone drives them anywhere. Ours asks for a detector by name, and if it does not have one, the swarm makes its own training pictures, trains a detector on the Radeon, and sends it to the other drones mid-mission.

2. How it works

Part	What it is	Runs on
Reasoning	Gemma 4 E4B through llama.cpp, one slot per drone	**W7900**
Training	Builds a new YOLO26 detector on demand	**W7900**
Detection	YOLO26, one model per drone	**W7900**
Simulation	MuJoCo, bumpy ground, crop rows, real quadrotor physics	**W7900**
Flying	Position controller, then steering by camera on the way down	CPU
Radio	Zenoh, one session per drone	CPU

The drones hold a line to within **6 cm**. Coming down, they do not fly to a saved coordinate. They steer by what the camera sees, and only keep descending while the weed stays in the middle of the frame.

Each drone keeps its own private map of where the weeds are. Nothing is shared by reference. A drone learns about a weed by seeing it or by being told, and two drones are allowed to disagree. When both want the same weed, the first claim wins. Detector weights travel as real bytes, chunked and checksummed.

You can prove that by breaking it. Switch a drone off in mid-air and the others notice the silence, take back the weeds it had claimed, and one of them re-plans the ground it never reached.

What is *not* distributed: the physics is one program, and the radio runs over loopback. So none of this tells you anything about real radio range.

3. Training data nobody labelled

No human drew a box, and no auto-labelling model was used.

The pictures are rendered in the simulator, and the labels come from the simulator's own geometry. A second pass paints every pixel with the id of the object it belongs to, so each box is exact. A weed hidden behind a leaf has no pixels. A weed half out of frame gets clipped properly.

We first did it the usual way, projecting where each weed should be and casting a ray to check. On one test view the new method found **20 weeds where the old one found 13**. Those seven would have been taught to the detector as empty ground.

The set is 3,380 pictures and 29,018 boxes, from 26 field layouts with the size, spacing, crop height and weed count all varied. It renders at about **19 pictures a second**.

4. What the Radeon is doing

Four jobs share the card at once.

Job	Measured	GPU
Gemma 4 thinking	**100.1 tokens/s**	96-97%
Training a detector	9-12 epochs, 60-140 s	85-100%
Detection	**8.1-8.3 ms a frame (120-123 fps)**	20-46%
Rendering 5 cameras	**10.2 ms** a round	

The detector reaches **0.9922 mAP@50**, with the card peaking at 94%. The card itself does 20.5 TFLOPS fp32 and 81.9 fp16.

We got an accidental CPU comparison. A dependency mix-up installed a CPU build of torch and training carried on without the GPU, silently. Same data, same settings, same machine: 6.8 seconds per step on CPU against 2.0 steps per second on the W7900. **13.6 times faster.** It cost us hours to spot, which is why we can quote it honestly.

Rendering was the bottleneck, not physics. Every control decision needs a fresh frame, so the simulation only runs as fast as the pictures.

Size	W7900	CPU
320 x 240	**83.7 fps**	0.8 fps
640 x 480	**83.8 fps**	1.3 fps

Drawing happened once per camera, one after another. Now each thread owns its own renderer, which is the only legal way to do it, because an OpenGL context belongs to one thread at a time.

Five cameras	Time	Rate
One at a time	63.0 ms	15.9 rounds/s
One renderer per thread	**10.2 ms**	**98.2 rounds/s**

Five cameras now cost about what one used to, **6.2 times faster**. A three-drone mission runs at roughly real time.

5. Learning to see, mid-mission

This is the part that is not a scripted flight.

The order goes to all the drones at once and each reads it with its own copy of Gemma. Three drones agreed on the target in **5.7 seconds**, against 15.7 if asked one at a time. Then all three say the same thing: they cannot see that plant.

So one of them makes the training data. It renders **800 labelled pictures** of that species from the heights a drone actually flies, taking about 70 seconds with the card at 95%, trains a detector, and sends **5.38 MB in 83 chunks** over the radio. The others rebuild it, check it and load it.

Then it checks how well it can really see. An accuracy figure is an average over every height the detector trained on, but a survey happens at one height. So the swarm tests itself. For common ragweed, thin and upright and easy to confuse with the crop, it measured 33% at 8 m, 46% at 6.5 m and **69% at 5 m**, and chose 5 m.

Nothing is sprayed until the plant is checked again from under 3 m. The survey sees it from 8 m, the dive sees it from 2 m, and the closer look wins. If they disagree, the drone pulls out and sprays nothing.

6. Results

Four randomly generated fields, three drones, the same order every time: "*spray the canada thistle weeds in the field, and leave the yellow mustard flowers alone*". The simulator knows where every plant is, so these are scored against the truth. A spray counts only if it lands within 1 m of a real weed that has not already been counted.

Field	Thistles	Recall	Precision	Coverage	Mustard hit
101	12	91.7%	100%	100%	0
108	6	100%	100%	100%	0
115	5	60.0%	75.0%	100%	1
122	8	62.5%	83.3%	100%	2
Average	7.8	**78.5%**	**89.6%**	**100%**	**0.75**

Read the range, not the best field: recall 60-100%, precision 75-100%. Four fields is a small sample. The weak ones are the sparse fields, where there are only five weeds to find and missing one costs 20 points.

Recall is the weaker number: about one weed in five is missed. And the swarm **does still spray plants it was told to protect**, 0.75 per field out of roughly 16, though two of the four fields came through clean. The close-range check refuses when it sees the wrong plant, but when it cannot make out anything at all it neither confirms nor refuses, and the spray goes ahead.

The point is the chemical. Spraying the whole field covers 256 m2 to deal with a dozen weeds. AgriSwarm puts a 0.45 m disc on each one, which is **2.0% of the field, a 98% reduction**, while removing 78.5% of the

weeds.

7. Where it falls short

- **It is all simulation.** No field trial, and we do not claim it transfers to

real plants.

- The detector is trained on pictures of the same simulator it is tested in.
- The physics is one program, and the radio is loopback.
- Wind, battery drain and losing a drone are built and tested, but were **not**

switched on for these runs.

- One weed in five is missed, and about 0.75 protected plants per field still

get sprayed.

- The Dockerfile has **never been built**, because neither machine had a

container runtime. The path we verified is `scripts/setup_env.sh`.

8. Running it

```
bash scripts/setup_env.sh /opt/agriswarm-venv     # verified installer
SLOTS=6 bash scripts/serve_gemma.sh               # Gemma on the GPU
bash scripts/ensemble.sh 4                        # 4 seeded fields
python -m pytest -q tests/                        # 68 tests
```

Every field comes from a seed, so runs repeat. Raw results are in `artifacts/ensemble.json`, and the README covers setup properly.

9. This is not really about farming

The weeds are the demonstration. The loop underneath, *say what you need to see, get a detector for it, share it*, has nothing to do with agriculture.

The extension point is one method, `acquire(target_class)`. What ships here serves detectors from the local Radeon. A production service such as **Aionvis** (aionvis.com) would sit behind the same interface. **It is not wired in here**, and everything measured in this report ran on the one W7900.